FULL STACK DEVELOPER PATH

Chapter 3 React Fundamentals

Declarative · Composable · UI = f(data)

IMPERATIVE — vanilla JS

```
createElement("h1")
createTextNode("Hi")
h1.appendChild(text)
root.appendChild(h1)
```

tell it every single step

vs

DECLARATIVE — React

```
<h1>Hi</h1>
```

just describe the result —
React handles the how
"please make me a sandwich"

SOURCE : freeCodeCamp × Scrimba — "Become a Fullstack Developer from Scratch"

MODULES : Static pages (26:30:04) · Data driven (28:48:33) · React state (31:03:00) · Side effects (36:17:02)

COVERS : JSX · components · props · mapping · state · forms · effects

PROJECTS : React Facts · Travel Journal · Chef Claude

⚡ WHY REACT

React is declarative and composable. You describe what the page should look like for a given set of data, and React works out the DOM operations. Build small pieces, combine them into something **greater than the parts.**

1 Rendering your first React ★★★★★

```jsx
// index.jsx
import { createRoot } from "react-dom/client";

const root = createRoot(document.getElementById("root"));
root.render(<h1>This is React</h1>);
```

- `createRoot` is told *where* to put things — a DOM node.
- That node is the `<div id="root">` in your HTML.
- `.render()` is told *what* to put there.

⚠ USE THE .jsx EXTENSION

Any file containing JSX should be `.jsx`, not `.js`. Vite recommends this — it helps it bundle your code performantly.

2 What JSX really is ★★★★★

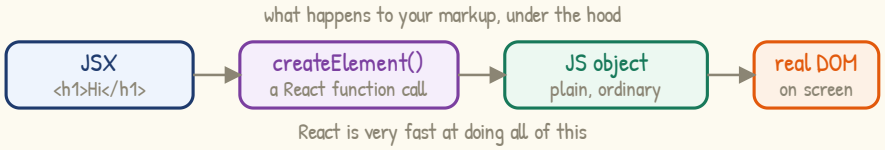


Fig 1 — JSX is compiled, not magic

DEFINITION

JSX looks like HTML but is compiled into calls to `React.createElement`, which return plain JavaScript objects that React interprets and turns into real DOM nodes.

▶ One parent element only

✗ Two siblings side by side

```jsx
root.render(
  <img />
  <h1>Hello</h1>
); // ✗ error
```

✓ Wrapped in one parent

```jsx
root.render(
  <div>
    <img />
    <h1>Hello</h1>
  </div>
);
```

⚠ THE ERROR YOU WILL SEE

JSX expressions must have one parent element. Think of it as trying to call two functions back to back on one line. The parent may have as many children as you like — but there must be exactly one root.

3 Fragments ★★★★

Wrapping in a `<div>` works, but inserts a **nested div whose purpose doesn't really exist**. React's own answer is the *fragment*.

```jsx
import { Fragment } from "react";

<Fragment>
    <img />
    <h1>Hello</h1>
</Fragment>
```

```jsx
// shorthand — no import needed
<>
    <img />
    <h1>Hello</h1>
</>
```

WHY IT'S BETTER

A fragment groups elements without creating another DOM node. Cleaner output, and it avoids odd side effects with **flexbox and grid**, where a stray wrapper div breaks the parent–child relationship.

4 Custom components ★★★★★

DEFINITION

A **React component** is simply a function that returns React elements. (You'll also hear "returns UI" — same thing.)

```jsx
function Header() {
    return (
        <header>
            <img src="logo.png" />
        </header>
    );
}

// render an INSTANCE of it
root.render(<Header />);
```

Rule	Why
PascalCase	Capital first letter — `Header`, not `header`
Angle brackets	`<Header />`, never `Header()`
Parentheses	Let the JSX sit on its own lines

⚠ TWO CLASSIC MISTAKES

1. Lowercase name — React won't treat it as a component.
2. Calling it like a function. `<Header />` tells React to create an instance of the component; that is what "render" means here.

⚡ COMPOSABILITY

Break a big UI into small pieces, make each flexible enough to be reused, and you write less code — which leads to **fewer bugs** — rather than trying to carve the final product from one giant block of stone.

5 One component per file ★★★★

```jsx
// Header.jsx
export default function Header() {
    return <header>...</header>;
}
```

```jsx
// index.jsx
import Header from "./Header";

// no curly braces — it's a default
```

Detail	Meaning
`export default`	Send one main thing out of the file
`./` prefix	My own file — **not** a package in `node_modules`
Extension	Optional; `.js` is assumed, `.jsx` usually resolves too
Naming	A default import can be called anything — keep it consistent anyway

Most teams collect these into a `components/` folder. Conventions vary by project — "don't take what I'm doing here as gospel."

6 Props ★★★★★

DEFINITION

Props pass data from a **parent** component down to a **child**. Conceptually very similar to attributes in HTML.

```jsx
// parent passes them in
<Joke
  setup="How did the CSS feel?"
  punchline="Depressed."
  upvotes={10}
  isPun={true}
/>
```

```jsx
// child receives one object
function Joke(props) {
    return (
        <div>
          <p>{props.setup}</p>
        </div>
    );
}
```

⚠ STRINGS vs EVERYTHING ELSE

Quotes `" "` pass a **string**. Curly braces `{ }` switch into JavaScript mode and pass the real value — a number, boolean, array or object. `upvotes={10}` is the **number** 10, so you can do maths on it; `upvotes="10"` is text.

▶ Destructuring props

```jsx
function Joke({ setup, punchline }) {   // pull them straight out
    return <p>{setup}</p>;              // no more props. prefix
}
```

PASSING THE WHOLE OBJECT

When a child wants most properties of an object, don't spell out 20 separate props — pass the object itself: `<Entry entry={entry} />`. You choose the prop name.

7 Rendering lists with .map()

★★★★★

React knows how to render an array — provided it is an array of JSX elements. Combine that with `.map()` and your display becomes a function of your data.

```
const entryElements = entries.map(entry => (
    <Entry
        key={entry.id}
        entry={entry}
    />
));

return <main>{entryElements}</main>;
```

⚠ OBJECTS ARE NOT VALID AS A REACT CHILD

A very common error. React cannot render a plain object — only the special objects created by JSX. Map your raw data into JSX first.

▶ The key prop

⚠ IT MUST BE CALLED key

You normally choose your own prop names — this is the exception. The warning reads: each child in a list should have a unique "key" prop.

Key source	Verdict
Database `id`	Best — guaranteed unique
A text field	Risky — repeats happen
Array `index`	Avoid — works in a pinch
Hard-coded `1`	Never — not unique

WHY REACT NEEDS IT

React uses keys to keep track of which item is which when items are added or removed from a list. Without a stable key it cannot tell what actually changed.

8 Conditional rendering

★★★★★

Show it only if it exists — `&&`

```
{props.setup &&
    <p>{props.setup}</p>
}
```

Choose between two — ternary

```
{isFavorite
    ? "Remove favourite"
    : "Add favourite"}
```

⚡ IT'S JUST JAVASCRIPT

Inside `{ }` you can run any JavaScript expression that returns a value. That's why the ternary and `&&` from Chapter 2 work unchanged here — React leans on plain JS rather than inventing its own template language.

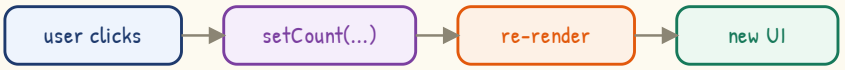
9 useState ★★★★★

DEFINITION

Props are data passed in from a parent and are **read-only**. *State* is data a component owns and can change — and changing it makes React re-render.

```
const [count, setCount] = React.useState(0);
      ▲       ▲                          ▲
   the value  the setter                 initial value
```

- `useState` returns an **array** — destructure it immediately.
- By convention the setter is the state name prefixed with **set**.
- Calling the setter updates state and triggers a re-render.



state changes → React redraws → the screen matches the data

Fig 2 — the React data flow loop

▶ Event handlers

```
function App() {
   const [count, setCount] = React.useState(0);

   function add() {          // declare handlers INSIDE the
      setCount(count + 1);   // component, above the return
   }

   return <button onClick={add}>+</button>;
}
```

⚠ A HARD-CODED SETTER IS A DEAD END

Calling `setIsImportant("definitely")` works once — but clicking again sets the same value, so nothing changes. To keep changing, the new value must be **derived from the old one**.

⚠ onClick, NOT onclick

JSX uses **camelCase** for event handlers. Pass the function by name — `onClick={add}`. Writing `onClick={add()}` calls it immediately during render instead of on click.

10 Never mutate state ★★★★★

⚠ THE BIGGEST BEGINNER MISTAKE

`count++` is the same as `count = count + 1` — it modifies the variable directly. In React that is a **big no-no**. If you are ever modifying the value directly, you're doing it wrong.

Attempt	Result
`count++` with `const`	*Error:* assignment to constant variable
Switching to `let`	Odd behaviour — only updates every second click
`++count`	"Works", but still wrong — don't get used to it
`setCount(count + 1)`	*Correct* — supplies a new value

The `const` warning was the first red flag. `count + 1` doesn't modify `count` — it just **describes** the new value.

11 Two ways to set state ★★★★★

1 — a plain new value

```
setIsImportant("definitely");
```

Use when you don't care what the old value was.

2 — a callback function

```
setCount(prevCount => prevCount + 1);
```

Use when the new value depends on the old one.

HOW THE CALLBACK WORKS

React passes the **old version of state** in as a parameter, and whatever the callback **returns** becomes the new state.

▶ Arrays and objects in state

```
// add to an array — never .push()
setIngredients(prevIngredients => [...prevIngredients, newIngredient]);

// flip a boolean
setIsGoingOut(prev => !prev);
```

⚠ UPDATING AN OBJECT IN STATE

Same rule: spread the old object into a new one, then override the field you want. `setContact(prev => ({ ...prev, isFavorite: !prev.isFavorite }))`. Note the parentheses around the object — otherwise JS reads `{` as a function body.

⚡ IMMUTABILITY

Props are immutable, and state is treated as immutable too. You never set state equal to something else — behind the scenes React **replaces state with a brand-new version**, which is what causes the component to re-run. Spread the old array into a new one rather than pushing into it.

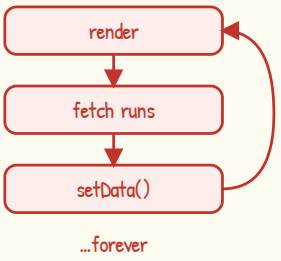
12 The infinite loop ★★★★★

Fetching data needs two steps: **get the data**, then **save it in state** so it can be displayed. Doing that directly in the component body creates a classic React bug.

```
// ✗ naive — infinite loop
function App() {
  const [data, setData] = React.useState(null);

  fetch("https://swapi.dev/api/people/1")
    .then(res => res.json())
    .then(data => setData(data));

  return <pre>{...}</pre>;
}
```



⚠ WHY IT LOOPS

Setting state **re-renders the component**. Re-rendering runs all the code in the body again — including the fetch — which sets state, which re-renders... You may not even see it happening.

DEFINITION — side effect

Anything that reaches outside your React app: fetching data, writing to local storage, subscribing to a websocket. Components should **avoid side effects** — imagine a component that added a database row every render.

13 useEffect ★★★★★

```
React.useEffect(() => {
  fetch(url).then(res => res.json()).then(data => setData(data));
}, [count]);
   ▲
dependencies array
```

⚡ PURE COMPONENTS

A component should behave like a **pure function**: same input, same output, no reaching outside. `useEffect` is the deliberate **escape hatch** for the times you must.

THE TWO PARAMETERS

1 — a callback function, usually an **inline anonymous arrow function**. By default it runs on every single render.

2 — the **dependencies array**, which stops that. In practice you will pretty much always include it.

14 How dependencies work ★★★★★

React stores the array between renders and compares it to the next one. If any value changed, the effect runs again. If nothing changed, it is skipped.

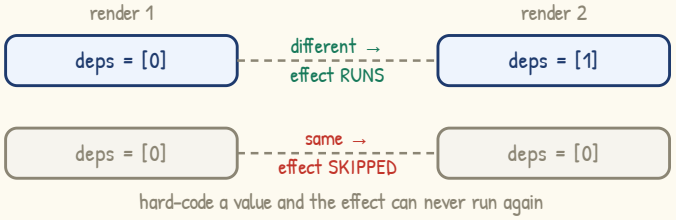


Fig 3 — React compares the array between renders

Written as	When the effect runs
`useEffect(fn)`	Every single render (rarely what you want)
`useEffect(fn, [])`	Once, on the first render only
`useEffect(fn, [count])`	First render, then whenever `count` changes

⚠ HOW THE LOOP GETS FIXED

With a dependencies array, setting state still re-renders the component — but the effect does not re-run, because nothing it depends on changed. No second fetch, so no loop.

▶ Worked example — why count was the wrong dependency

Deps array	What actually happens
`[count]`	Re-fetches every time the counter is clicked — wasteful
`[0]` hard-coded	Arrays match forever, so the effect never runs again
`[]` empty	Fetch once when the component first appears

Put a value in the array only if the effect genuinely **depends** on it. A counter that has nothing to do with the request does not belong there.

⚡ REASONING ABOUT ORDER

On first load: the component body runs → React paints the page → then the effect runs. That is why data-fetching state usually starts as `null` and the page renders empty for a moment before the data lands.

15 Dynamic styling ★★★★

Because styles are just data, they can be driven by state or props — the same declarative idea applied to appearance.

▶ Inline style objects

```
const styles = {
  backgroundColor: darkMode ? "#1e1e1e" : "#f5f5f5"
};

<button style={styles}>...</button>
<button style={{ backgroundColor: props.color }}>...</button>
            ▲ ▲
      outer { } = JavaScript · inner { } = the object
```

⚠ TWO SETS OF BRACES

*The **first** brace switches into JavaScript; the **second** is the object literal itself. The attribute must be spelled* `style` *— not* `styles` *— because it is a real property on the button element.*

▶ Dynamic class names

```
<button className={props.on ? "on" : ""}>...</button>
```

className, NOT class

In JSX the attribute is `className`*. Toggling a class is usually cleaner than inline styles — keep the look in your CSS file and let React decide only **which class applies**.*

16 Where should state live? ★★★★

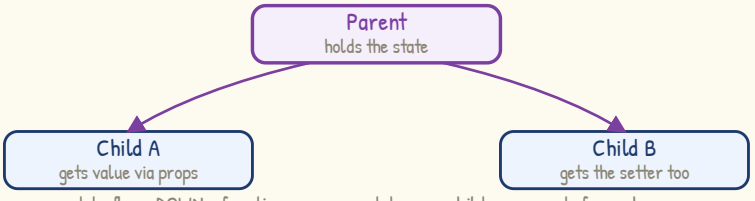


Fig 4 — state lives in the closest common parent

⚡ THE DECIDING QUESTION

*If a parent needs a value — say, to decide whether to render something — the state must live in the parent. Pushing it down into a child leaves **no good way to get the value back up**.*

*Pass the **setter function down as a prop** so the child can trigger the change.* Architecting React apps takes years of experience — there is rarely one right answer.

17 Project - Travel Journal ★★★★

Pure props and mapping: an array of entry objects becomes an array of components. The display is a function of the data — add an object to the array, and a new card appears on the page.

```jsx
// App.jsx
import entries from "./data";

export default function App() {
    const cards = entries.map(entry => (
        <Entry key={entry.id} entry={entry} />
    ));
    return <main>{cards}</main>;
}
```

STATIC IMAGES

A relative path works in Scrimba/Vite, but in a fresh Vite project it may not. The quick fix is an absolute path from the project root — e.g. `/src/assets/logo.png`.

18 Project - Chef Claude ★★★★

A form that collects ingredients into an array held in state.

```jsx
const [ingredients, setIngredients] = React.useState([]);

function handleSubmit(newIngredient) {
    setIngredients(prev => [...prev, newIngredient]);   // ✓ new array
    // ingredients.push(newIngredient)  ✗ never do this
}
```

⚠ THE ONE-LINE REFACTOR

Moving a plain array into state is a small change — delete the `.push()` *and call the setter with a* ***brand-new array*** *instead. The rest of the component is untouched, because the display already reads from that variable.*

KNOWN ROUGH EDGE

After submitting, the input ***does not clear itself*** *— the field is not yet wired to state. Handling that properly is what controlled form inputs solve, covered later in the course.*

⚡ QUICK REVISION

▶ Syntax cheat-sheet

Code	Meaning
`<Header />`	Render a component
`<> </>`	Fragment
`{ expression }`	JavaScript mode
`props.name`	Read a prop
`export default`	Send out of file

Code	Meaning
`useState(0)`	Create state
`setX(prev => ...)`	Update from old
`useEffect(fn, [])`	Run once
`onClick={fn}`	Event handler
`key={item.id}`	List identity

▶ Props vs state

	Props	State
Comes from	The parent component	The component itself
Can change?	No — read-only	Yes — via the setter
Mutable?	Immutable	Also treated as immutable

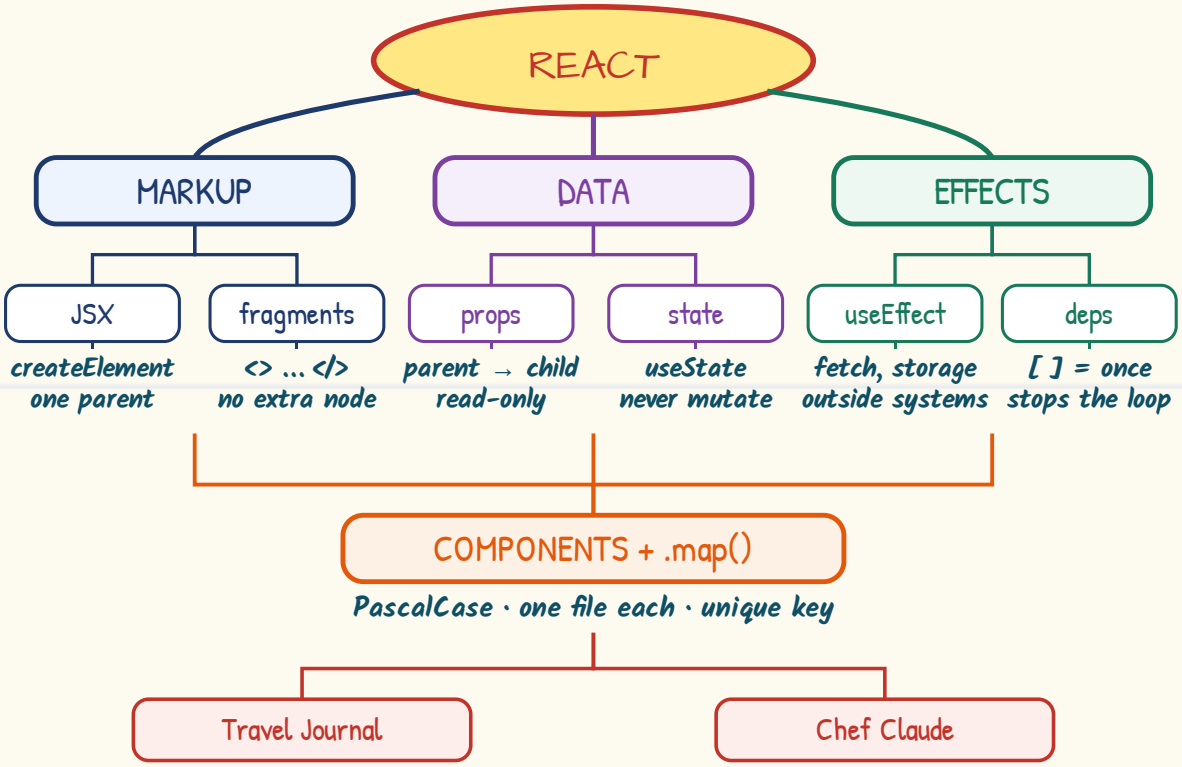
▶ 10 ultra-important exam facts

1. A component is **a function that returns React elements**.
2. Component names must be PascalCase, rendered as `<Name />`.
3. JSX compiles to `createElement` → plain JS objects → real DOM.
4. JSX must return **exactly one parent element**; a fragment adds no DOM node.
5. Curly braces pass real values; quotes pass strings.
6. Every mapped list item needs a **unique** `key` — prefer an `id`.
7. React cannot render plain objects — only JSX-created ones.
8. Never mutate state; always call the setter with a new value.
9. Use the callback form when the new state depends on the old.
10. Fetching in the component body causes an **infinite loop** — use `useEffect`.

▶ Common mistakes & traps

✗ Mistake	✓ Correct
`count++` in a handler	`setCount(prev => prev + 1)`
`ingredients.push(x)`	`setIngredients([...prev, x])`
`onClick={add()}`	`onClick={add}` — pass, don't call
`<header />` lowercase	`<Header />` PascalCase
`key={1}` or the index	`key={item.id}`
fetch in the component body	fetch inside `useEffect`

MIND MAP - CHAPTER AT A GLANCE



describe the UI for a given state — React does the rest

next up → capstone · TypeScript · Next.js

▶ When to reach for what

You want to...	Reach for
Reuse a chunk of UI	A **component** in its own file
Feed it data from outside	**props**
Remember something that changes	**useState**
Show a list	**.map()** + a unique **key**
Show something only sometimes	**&&** or a ternary
Talk to the outside world	**useEffect** + deps array

⚡ FINAL WORD

The whole model in one line: *UI is a function of state*. Change the data, and React re-renders whatever depends on it. If you find yourself reaching for **document.getElementById** inside a component, you are fighting the framework.